

Summary - LLMs Security & Cybesecurity

Introduction

This course offers a structured, operational introduction to the secure design and operation of applications powered by Large Language Models (LLMs). The ambition is twofold: (1) provide a clear conceptual frame for architectural and security decisions across the LLM stack, and (2) install measurement reflexes that make these decisions demonstrable and auditable. The constant thread is product-risk alignment: start from concrete security use cases (prompt injection defense, output handling, code/config review), select patterns and controls that fit the risk, implement them in small, inspectable systems, and observe their measurable signatures (block rates, false-positive/negative deltas, latency/cost overheads). Local execution and controlled testbeds are a deliberate choice to make learning reproducible, inspectable, and independent from external services.

Objectives and scope

Students learn to characterize the LLM security lifecycle and to distinguish model risks (content safety, data leakage, jailbreaks) from system risks (tool misuse, insecure plugins, excessive agency). They can explain the role and limits of prompts, guardrails, tool contracts, evaluation harnesses, and governance frameworks (e.g., NIST AI RMF). They understand how input sources, retrieval (RAG), and tools create attack surfaces, why output handling must be treated like untrusted input, and how these choices leave measurable traces in evaluation dashboards and logs. Coverage remains platform-centric and practice-driven: threat modeling, mitigations, measurement—without drifting into a course on ML algorithms.

Targeted competencies

By the end of the semester, students can:

- Elicit security requirements for LLM applications from real user journeys and threats.
- Design prompts and contexts that minimize risk; recognize and neutralize prompt injection patterns.
- Implement deny-by-default tool use with explicit, testable contracts and least privilege.

- - Treat model outputs as untrusted: validate against schemas, sanitize, and gate downstream execution.
- Plan and run security evaluations (prompt suites, red-teaming, replay), interpret metrics (block rate, FP/FN, cost/latency), and iterate.
- Document decisions and trade-offs with evidence: traces, evaluations, before/after metrics, and residual-risk rationale.

Pedagogical architecture

The progression follows six coherent blocks: (1) Threats & Taxonomies (LLM01–LLM10); (2) Prompt Injection & Output Handling; (3) Tooling & Plugins; (4) Retrieval & Data Poisoning; (5) Agency & Autonomy; (6) Governance & Measurement. Each session alternates guided exposition with a mini-demo centered on a measurable control (e.g., jailbreak suite vs guardrail variants; schema validation vs raw rendering; deny-by-default tools vs permissive agents). Labs extend these gestures on small, self-contained projects to reinforce the link between design choices and observed behavior.

Assessment

A zero-risk “Lab 0” secures environment setup and evidence-collection method (evals and logs). Three graded labs build difficulty: Lab 1 strengthens prompt/eval basics; Lab 2 implements secure tool invocation with contracts and telemetry; Lab 3 hardens a small RAG/agent by mitigating injection, insecure output handling, and excessive agency. The final project—conducted in two milestones—defines a use case and SLAs/SLOs, deploys a minimal but secure LLM workflow, then hardens it (guardrails, contracts, context minimization, evaluation flywheel), with quantitative evidence and clear documentation. Grading values traceability and the quality of trade-off arguments.

Technical environment

The stack is intentionally lean to foreground fundamentals: Python 3.10+, a local LLM or API gateway, evaluation harness (prompt suites + replay), lightweight guardrails (schema validation, allow/deny lists), and a simple telemetry layer for costs/latency/blocks. Projects run locally or in a contained sandbox to keep execution fast and iterations frequent. Students receive a measurement journal template and guidance on versioning configurations—conditions for straightforward reproducibility.

Expected outcomes

Upon completion, each student can design a small but well-justified secure LLM workflow, choose controls according to measurable objectives, read evaluation dashboards, identify dominant risks, and propose concrete improvements. They produce a concise security memo (threat model,

controls, evidence, residual risks) and can defend recommendations with indisputable indicators and traces. The skill set transfers directly to larger environments and equips subsequent projects (code review assistants, RAG apps, or tool-using agents) without losing the evidence-driven discipline that underpins trustworthy AI systems.

Key references for the course

- **OWASP Top 10 for LLM Applications** (latest status, risks LLM01 → LLM10 and mitigations).
- **MITRE ATLAS** (AI attack tactics/techniques and red teaming scenarios).
- **NIST AI RMF 1.0** (Govern • Map • Measure • Manage governance framework). NIST publications
- **Microsoft Defender for Cloud – AI security content** (telemetry, jailbreak/prompt injection/DoS detections).
- **RedTeam Arena** : challenge [plateform](#) for jailbreaking LLM models.